Delivery Versus Payment Audit Report

Version 1.0

Shadowy Creators

October 14, 2025

Delivery Versus Payment Audit Report

Shadowy Creators

October 14, 2025

Prepared by: Shadowy Creators Lead Auditors: - Mattia Papa - Alessandro Bergamaschi

Table of Contents

Protocol Summary

Delivery Versus Payment (DVP) is a permissionless protocol that enables atomic swaps of an arbitrary number of assets between multiple parties. The protocol supports native ETH, ERC-20 tokens, and ERC-721 NFTs in a single settlement. Users create settlements containing multiple flows (asset transfers), all involved parties must approve before execution, and transfers occur atomically - either all succeed or all fail. The protocol operates as a non-upgradeable singleton contract with no admin privileges, emphasizing decentralization and immutability.

Disclaimer

The Shadowy Creators team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond to the following commit hash:

```
e377b72c9e72e408fdb3c3c78de6ecac5a368574
```

Scope

- `DeliveryVersusPaymentV1.sol`
- `DeliveryVersusPaymentV1HelperV1.sol`
- `IDeliveryVersusPaymentV1.sol`

Roles

There are no admin roles or privileged accounts in the DVP protocol. The system is entirely permissionless where any address can create settlements, only involved parties can approve their own participation, and anyone can execute fully-approved settlements.

Issues found

Severity	Number of issues found
High	0
Medium	0
Low	2
Info	3

Severity	Number of issues found
Gas Optimizations	9
Total	14

Findings

High

None

Medium

None

Low

[L-1] ERC20 and NFT validation could be enhanced to reduce false positives and negatives

Description: The current token validation methods in `_isERC20()` and `_isERC721()` use minimal checks that may exclude legitimate tokens or accept invalid ones.

Current validation limitations:

1. ERC721 validation (lines 553-555):

```
function _isERC721(address token) internal view returns (bool) {
  return token.supportsInterface(type(IERC721).interfaceId);
}
```

- Exclusion issue: Legitimate NFTs that don't implement ERC165 are rejected
- False positives: Contracts that are not NFT can implement a ERC721 interface and pass the validation

1. ERC20 validation (lines 560-563):

```
function _isERC20(address token) internal view returns (bool) {
  (bool success, bytes memory result) = token.staticcall(abi.
     encodeWithSelector(SELECTOR_ERC20_DECIMALS));
```

```
    return success && result.length == 32;
}
```

- False positives: Any contract with `decimals()` function passes (oracles, config contracts, etc.)

Recommended enhancements:

1. Multi-function ERC20 validation:

```
function _isERC20(address token) internal view returns (bool) {
    bytes4 nameSelector = bytes4(keccak256("name()"));
    bytes4 symbolSelector = bytes4(keccak256("symbol()"));
    bytes4 balanceOfSelector = bytes4(keccak256("balanceOf(address)"));
    bytes4 decimalsSelector = bytes4(keccak256("decimals()"));

    uint256 validFunctions;
    bool success;

    (success,) = token.staticcall(abi.encodeWithSelector(nameSelector));
    if (success) validFunctions++;

    (success,) = token.staticcall(abi.encodeWithSelector(symbolSelector))
        ;
    if (success) validFunctions++;

    (success, bytes memory result) = token.staticcall(abi.
        encodeWithSelector(balanceOfSelector, address(this)));
    if (success && result.length == 32) validFunctions++;

    (success, result) = token.staticcall(abi.encodeWithSelector(
        decimalsSelector));
    if (success && result.length == 32) validFunctions++;

    return validFunctions == 4;
}
```

2. Fallback ERC721 validation (avoiding ERC165 dependency):

```
function _isERC721(address token) internal view returns (bool) {
    // First try ERC165 (standard method)
    try IERC165(token).supportsInterface(type(IERC721).interfaceId)
        returns (bool supports) {
      if (supports) return true;
    } catch {}

    // Fallback: Check for core ERC721 functions (for non-ERC165 NFTs)
    bytes4 ownerOfSelector = bytes4(keccak256("ownerOf(uint256)"));
    bytes4 balanceOfSelector = bytes4(keccak256("balanceOf(address)"));

    uint256 validFunctions;
```

```
12    bool success;
13
14    (success,) = token.staticcall(abi.encodeWithSelector(ownerOfSelector,
          1));
15    if (success) validFunctions++;
16
17    (success, bytes memory result) = token.staticcall(abi.
          encodeWithSelector(balanceOfSelector, address(this)));
18    if (success && result.length == 32) validFunctions++;
19
20    return validFunctions == 2;
21  }
```

Benefits:

- Reduced false positives: Multi-function validation prevents non-token contracts from being accepted

Trade-offs:

- Higher gas costs: Additional validation calls (~10,000-15,000 gas per token)
- Increased complexity: More sophisticated validation logic
- Still not perfect: Cannot guarantee 100% accuracy without calling actual token functions

[L-2] Unbounded loops enable gas-based denial of service

Description: Multiple functions contain unbounded loops over flows array and settlement IDs arrays without gas limit considerations. Large settlements or batch operations can exceed block gas limits.

Risks:

- Settlements with too many flows become unexecutable within block gas limits
- Batch operations on many settlements can fail due to gas exhaustion
- Griefing: malicious actors can create settlements with excessive flows to waste gas of executors
- Legitimate large settlements may become permanently unexecutable

Recommended Mitigation:

- Consider adding soft caps on number of flows per settlement
- Document gas considerations clearly for users
- Implement pagination for batch operations where feasible

Acknowledgement:

- Known issue documented in README.md lines 121-122: “The current chain’s block gas limit acts as a cap. In every case it is the caller’s responsibility to ensure that the gas requirement can be met.”

Informational

[I-1] Inconsistent error handling between executeSettlement and auto-execution

Description: The contract uses different error handling patterns for manual settlement execution versus auto-execution, leading to inconsistent logging and user experience. - Auto-execution failures are caught and logged with detailed error events - Manual execution failures bubble up as regular reverts with no additional logging This creates different debugging experiences for the same underlying operation.

Current implementation: Auto-execution in `approveSettlements()` (lines 231-232):

```
try this.executeSettlementInner(msg.sender, settlementId) {
} catch Error(string memory reason) {
    emit SettlementAutoExecutionFailedReason(settlementId, msg.sender,
        reason);
} catch Panic(uint errorCode) {
    emit SettlementAutoExecutionFailedPanic(settlementId, msg.sender,
        errorCode);
} catch (bytes memory lowLevelData) {
    emit SettlementAutoExecutionFailedOther(settlementId, msg.sender,
        lowLevelData);
}
```

Manual execution in `executeSettlement()` (line 292):

```
function executeSettlement(uint256 settlementId) external nonReentrant
    {
    this.executeSettlementInner(msg.sender, settlementId); // No try/
        catch
}
```

Recommended improvement:

Add consistent error handling to `executeSettlement()`:

```
function executeSettlement(uint256 settlementId) external nonReentrant
    {
    try this.executeSettlementInner(msg.sender, settlementId) {
        // Success - settlement executed
    } catch Error(string memory reason) {
```

```
5           emit SettlementExecutionFailedReason(settlementId, msg.sender,
                reason);
6           revert(reason); // Still revert for manual execution
7       } catch Panic(uint errorCode) {
8           emit SettlementExecutionFailedPanic(settlementId, msg.sender,
                errorCode);
9           // Re-throw panic
10          assembly { invalid() }
11      } catch (bytes memory lowLevelData) {
12          emit SettlementExecutionFailedOther(settlementId, msg.sender,
                lowLevelData);
13          // Re-throw the original error
14          assembly { revert(add(lowLevelData, 0x20), mload(lowLevelData))
                }
15      }
16  }
```

[I-2] Consider adding input validation for flow parameters

Description: The `createSettlement()` function could benefit from basic input validation to prevent obviously invalid settlements and improve user experience by catching errors early.

Current implementation: No validation is performed on flow parameters beyond token type validation.

Recommended validation:

```
1  // Validate flows
2  for (uint256 i = 0; i < lengthFlows; i++) {
3    Flow calldata flow = flows[i];
4
5    // Validate addresses for both ERC20 and NFTs
6    if (flow.from == address(0)) revert InvalidFromAddress();
7    if (flow.to == address(0)) revert InvalidToAddress();
8
9    // Validate amount/id based on token type
10   // For ERC20/ETH, zero amounts don't make sense; For NFTs, token ID 0
         is valid per ERC721 standard
11   if (!flow.isNFT && flow.amountOrId == 0) revert InvalidAmountOrId();
12 }
```

- Early error detection: Catches invalid parameters at creation time rather than execution time
- Better user experience: Clear error messages for common mistakes
- Gas savings: Prevents users from creating settlements that will never be executable
- Protocol robustness: Reduces likelihood of invalid settlements in the system

Note: This validation respects the ERC721 standard where token ID 0 is perfectly valid, while preventing

meaningless zero amounts for ERC20 tokens and ETH transfers.

[I-3] Front-running risk for settlement execution

Description: Malicious actors can front-run settlement execution by revoking approvals or modifying settlement state just before execution, causing the executor to waste gas on failed transactions.

Attack scenario:

1. A settlement is fully approved and ready for execution
2. User A calls `executeSettlement(settlementId)`
3. Malicious User B observes the transaction in the mempool
4. User B front-runs with `revokeApprovals([settlementId])` using higher gas price
5. User A's execution transaction fails with `SettlementNotApproved()` error
6. User A wastes gas on the failed execution attempt

Recommended mitigation:

- Private mempools: Use private transaction pools (e.g., Flashbots Protect) for `executeSettlement()` calls to avoid mempool visibility

Note: This is an inherent limitation of public blockchain execution rather than a contract vulnerability. The attacker also pays gas costs, making this primarily a griefing attack rather than a profitable exploit.

Gas

[G-1] Cache array length in loops to save gas

Description: In `isSettlementApproved()` function, `settlement.flows.length` is accessed multiple times but could be cached to save gas.

Current implementation (lines 144-146):

```
1 if (settlement.flows.length == 0) revert SettlementDoesNotExist();
2
3 uint256 lengthFlows = settlement.flows.length;
```

Recommended optimization:

```
1 uint256 lengthFlows = settlement.flows.length;
2 if (lengthFlows == 0) revert SettlementDoesNotExist();
```

Gas savings: Eliminates one SLOAD operation by caching the length before the conditional check.

[G-2] Optimize party address handling in loops

Description: In `isSettlementApproved()` function, the `party` variable declaration can be optimized.

Current implementation (lines 148-149):

```
address party = settlement.flows[i].from;
if (!settlement.approvals[party]) {
```

Alternative optimizations: 1. Declare outside loop:

```
address party;
for (uint256 i = 0; i < lengthFlows; i++) {
  party = settlement.flows[i].from;
  if (!settlement.approvals[party]) {
```

2. Use direct access (most gas efficient):

```
for (uint256 i = 0; i < lengthFlows; i++) {
  if (!settlement.approvals[settlement.flows[i].from]) {
```

Gas savings: Option 1 saves variable declaration gas in each loop iteration. Option 2 eliminates the temporary variable entirely, providing maximum gas efficiency.

[G-3] Optimize Settlement struct storage layout

Description: The `Settlement` struct can be optimized to reduce storage slot usage by reordering fields and using smaller data types.

Current implementation (lines 118-126):

```
struct Settlement {
  string settlementReference;
  uint256 cutoffDate;
  Flow[] flows;
  mapping(address => bool) approvals;
  mapping(address => uint256) ethDeposits;
  bool isSettled;
  bool isAutoSettled;
}
```

Recommended optimization:

```
struct Settlement {
  string settlementReference;
  Flow[] flows;
```

```
    mapping(address => bool) approvals;
    mapping(address => uint256) ethDeposits;
    uint128 cutoffDate;
    bool isSettled;
    bool isAutoSettled;
}
```

Gas savings:

- Reduces storage usage by packing fields into fewer storage slots
- `uint128` provides sufficient range for timestamps (valid until year ~10^31)
- **Settlement creation**: Saves gas by reducing storage operations
- **Settlement reads**: Saves gas when reading `cutoffDate` with boolean fields by reducing SLOAD operations
- Affected functions: `approveSettlements()`, `executeSettlementInner()`, `getSettlement()`,`withdrawETH()`
- **No modification savings**: These fields are never modified together after creation
- Optimization moves `cutoffDate` into the same storage slot as the boolean fields

[G-4] Use assignment instead of addition for ETH deposits

Description: In `approveSettlements()` function, ETH deposits use += operator when simple assignment would suffice and be more gas efficient.

Current implementation (line 202):

```
settlement.ethDeposits[msg.sender] += ethAmountRequired;
```

Recommended optimization:

```
settlement.ethDeposits[msg.sender] = ethAmountRequired;
```

Safety analysis:

- The function prevents double approvals with `if (settlement.approvals[msg.sender])revert ApprovalAlreadyGranted();`
- Since users cannot approve the same settlement twice, `ethDeposits[msg.sender]` is always 0 before the first (and only) approval
- Therefore `= ethAmountRequired` and `+= ethAmountRequired` produce identical results

Gas savings: Eliminates one SLOAD operation (~200 gas) by avoiding the need to read the current value before addition.

[G-5] Cache ETH deposit amount to avoid duplicate storage reads

Description: In `withdrawETH()` function, `settlement.ethDeposits[msg.sender]` is read twice but could be cached to save gas.

Current implementation (lines 439-441):

```
if (settlement.ethDeposits[msg.sender] == 0) revert NoETHToWithdraw();
uint256 amount = settlement.ethDeposits[msg.sender];
```

Recommended optimization:

```
uint256 amount = settlement.ethDeposits[msg.sender];
if (amount == 0) revert NoETHToWithdraw();
```

Gas savings: Eliminates one SLOAD operation (~2,100 gas) by caching the deposit amount before the conditional check.

[G-6] Add batch withdrawal functionality for consistency and gas efficiency

Description: The `withdrawETH()` function only accepts a single settlement ID, while other functions like `approveSettlements()` and `revokeApprovals()` support batch operations. Adding batch functionality would improve gas efficiency and user experience.

Current implementation:

```
function withdrawETH(uint256 settlementId) external nonReentrant
```

Recommended enhancement:

```
function withdrawETH(uint256[] calldata settlementIds) external
    nonReentrant {
     uint256 totalAmount = 0;

     for (uint256 i = 0; i < settlementIds.length; i++) {
         uint256 settlementId = settlementIds[i];
         Settlement storage settlement = settlements[settlementId];

         if (settlement.flows.length == 0) revert SettlementDoesNotExist
             ();
         if (block.timestamp <= settlement.cutoffDate) revert
             CutoffDateNotPassed();
         if (settlement.isSettled) revert SettlementAlreadyExecuted();

         uint256 amount = settlement.ethDeposits[msg.sender];
         if (amount > 0) {
             settlement.ethDeposits[msg.sender] = 0;
             totalAmount += amount;
```

```
            emit ETHWithdrawn(msg.sender, amount);
        }
    }

    if (totalAmount == 0) revert NoETHToWithdraw();
    Address.sendValue(payable(msg.sender), totalAmount);
}
```

- Gas efficiency: Amortizes transaction costs across multiple withdrawals
- User experience: Allows users to withdraw from multiple expired settlements in one transaction
- Consistency: Matches the batch operation pattern used by `approveSettlements()` and `revokeApprovals()`
- Flexibility: Users can still withdraw from single settlements by passing a single-element array

[G-7] Optimize revokeApprovals for batch ETH transfers and variable declarations

Description: The `revokeApprovals()` function can be optimized in two ways: batching ETH transfers into a single send operation and declaring loop variables outside the loop.

Current implementation:

```
for (uint256 i = 0; i < lengthSettlements; i++) {
    uint256 settlementId = settlementIds[i]; // Declared inside loop
    // ... validation ...
    uint256 ethAmountToRefund = settlement.ethDeposits[msg.sender];
    if (ethAmountToRefund > 0) {
        settlement.ethDeposits[msg.sender] = 0;
        Address.sendValue(payable(msg.sender), ethAmountToRefund); //
           Multiple sends!
        emit ETHWithdrawn(msg.sender, ethAmountToRefund);
    }
}
```

Recommended optimization:

```
uint256 totalRefund = 0;
uint256 settlementId; // Declared outside loop

for (uint256 i = 0; i < lengthSettlements; i++) {
    settlementId = settlementIds[i];
    Settlement storage settlement = settlements[settlementId];

    if (settlement.flows.length == 0) revert SettlementDoesNotExist();
    if (settlement.isSettled) revert SettlementAlreadyExecuted();
    if (!settlement.approvals[msg.sender]) revert ApprovalNotGranted();

    uint256 ethAmountToRefund = settlement.ethDeposits[msg.sender];
```

```
        if (ethAmountToRefund > 0) {
            settlement.ethDeposits[msg.sender] = 0;
            totalRefund += ethAmountToRefund;
            emit ETHWithdrawn(msg.sender, ethAmountToRefund);
        }

        settlement.approvals[msg.sender] = false;
        emit SettlementApprovalRevoked(settlementId, msg.sender);
    }

    // Single ETH transfer at the end
    if (totalRefund > 0) {
        Address.sendValue(payable(msg.sender), totalRefund);
    }
```

Gas savings:

- **Batch ETH transfers**: ~21,000 gas × (n-1) where n = number of settlements with ETH deposits
- **Variable declaration**: ~50-100 gas × number of settlements processed
- **Example**: For 5 settlements with ETH deposits, saves ~84,000+ gas from batching transfers alone

Additional benefits:

- **Reduced reentrancy surface**: Single external call instead of multiple
- **Consistency**: Matches the batch pattern used in `approveSettlements()`
- **Better UX**: Single transfer event instead of multiple small transfers

[G-8] Remove unnecessary return statement in getSettlementPartyStatus

Description: The `getSettlementPartyStatus()` function uses named return parameters but includes an unnecessary explicit return statement, wasting gas.

Current implementation:

```
function getSettlementPartyStatus(
    uint256 settlementId,
    address party
)
    external
    view
    returns (bool isApproved, uint256 etherRequired, uint256
       etherDeposited, TokenStatus[] memory tokenStatuses)
{
    Settlement storage settlement = settlements[settlementId];
    if (settlement.flows.length == 0) revert SettlementDoesNotExist();

    isApproved = settlement.approvals[party];
```

```
13      (etherRequired, etherDeposited) = _getPartyEthStats(settlement,
            party);
14      tokenStatuses = _getTokenStatuses(settlement, party);
15      return (isApproved, etherRequired, etherDeposited, tokenStatuses);
            // <-- Unnecessary!
16  }
```

Recommended optimization:

```
1   function getSettlementPartyStatus(
2       uint256 settlementId,
3       address party
4   )
5       external
6       view
7       returns (bool isApproved, uint256 etherRequired, uint256
            etherDeposited, TokenStatus[] memory tokenStatuses)
8   {
9       Settlement storage settlement = settlements[settlementId];
10      if (settlement.flows.length == 0) revert SettlementDoesNotExist();
11
12      isApproved = settlement.approvals[party];
13      (etherRequired, etherDeposited) = _getPartyEthStats(settlement,
            party);
14      tokenStatuses = _getTokenStatuses(settlement, party);
15      // No return statement needed - variables are already assigned!
16  }
```

- Eliminates the gas cost of the explicit return operation
- Small but consistent savings for this view function that may be called frequently
- Follows Solidity best practices for named return parameters

[G-9] Use unchecked increment in for loops to save gas

Description: All for loops in the contract use standard `i++` increment which includes overflow checks. Since loop counters are bounded by array lengths and cannot realistically overflow, using `unchecked` increment saves significant gas.

Current implementation pattern:

```
1   for (uint256 i = 0; i < lengthFlows; i++) {
2     // loop body
3   }
```

Recommended optimization:

```
1   for (uint256 i = 0; i < lengthFlows; ) {
2     // loop body
```

```
    unchecked {
      i++;
    }
  }
```

Affected functions:

- `createSettlement()` - flows validation loop
- `executeSettlementInner()` - flows execution loop

- `isSettlementApproved()` - flows approval check loop
- `approveSettlements()` - settlements loop
- `revokeApprovals()` - settlements loop
- `_getPartyEthStats()` - flows loop
- `_getTokenStatuses()` - flows loop

Gas savings:

- Eliminates overflow checks on loop increment operations
- Saves gas on every loop iteration across all functions
- Particularly beneficial for settlements with many flows or batch operations

Safety analysis:

- Loop counters are bounded by array lengths which cannot exceed reasonable limits
- No risk of overflow in practical usage scenarios
- Standard optimization pattern used throughout DeFi protocols

Architectural Improvements Proposal

Alternative Architecture: Meta-Signature Based Execution

Current Architecture:

The current DVP implementation follows a multi-transaction pattern: - Settlement creation and storage on-chain - Individual approval transactions from each party - Final execution transaction - All data and state maintained on-chain throughout the process

Alternative Approach:

An alternative architecture could leverage off-chain coordination with on-chain execution verification. Instead of storing approvals and settlement data on-chain, parties could coordinate off-chain and execute settlements atomically in a single transaction using cryptographic proofs of consent.

Potential Benefits:

- Significant gas reduction: Elimination of intermediate storage and multiple transactions
- Enhanced user experience: Streamlined single-transaction execution
- Improved scalability: More efficient handling of complex multi-party settlements
- Maintained security: Cryptographic verification ensures same trust guarantees

Considerations:

- Technical complexity: Requires sophisticated off-chain coordination mechanisms
- Infrastructure requirements: Need for reliable signature aggregation and coordination systems

Strategic Value:

This architectural approach represents a potential evolution path for the DVP protocol, offering substantial efficiency improvements while preserving the core atomic settlement guarantees. The implementation would require careful design of cryptographic verification mechanisms and off-chain coordination protocols.

Such architectural enhancements could position the protocol as a leading solution for efficient multi-party asset exchanges in the DeFi ecosystem.